

BLACK ACETATE / STUDY 01

Build these glasses in Three.js

An annotated construction blueprint, a working 3D model, and editable curve data.

Reconstruct the glossy black front, rounded rectangular lenses, integral bridge, silver oval rivets and curved flat arms from the supplied reference.

SOURCE PHOTOGRAPH



SUPPLIED THREE.JS RECONSTRUCTION



The visible design can be matched closely. Exact product size, hidden geometry and lens prescription cannot be established from one angled photograph. The orthographic drawings and dimensions in this guide are explicit estimates.

BLACK ACETATE / CONSTRUCTION BLUEPRINT

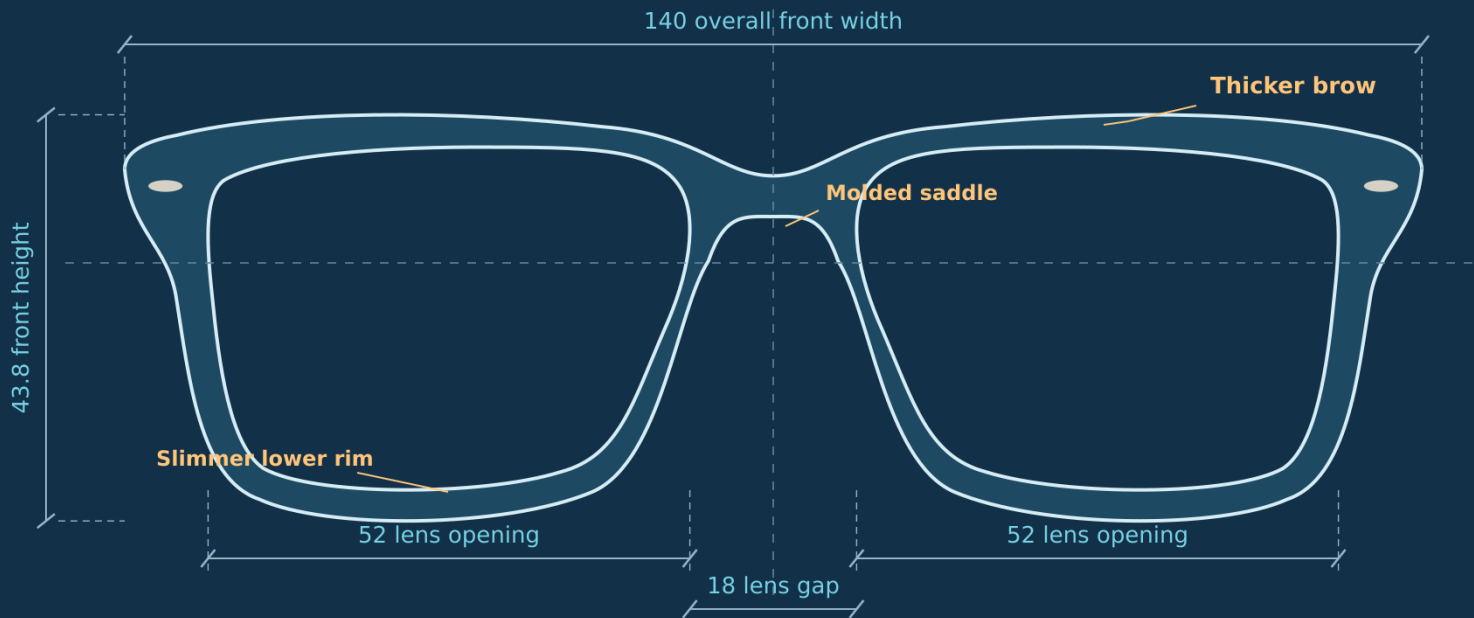
Reconstruct the visible design in Three.js · shared contour data · all dimensions are photo-derived estimates

STUDY 01

UNITS: mm

01 FRONT ELEVATION

One continuous acetate face · two shaped lens openings · symmetry assumed



REFERENCE / VISIBLE FEATURES

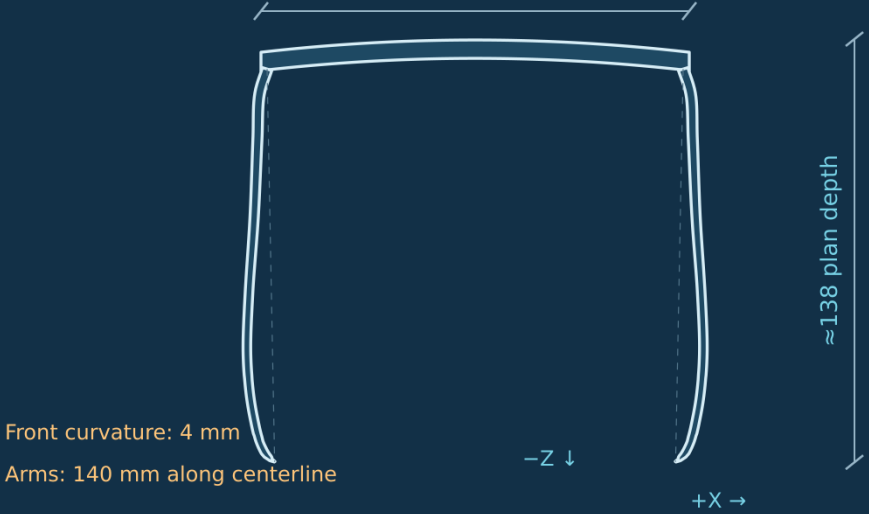
Original photograph establishes the appearance, not physical measurements



Glossy black acetate; rounded rectangular lenses
Silver oval face rivets; tapered, flattened arms
Clear lenses; integral bridge; downturned ear hooks
Hidden hinge and rear contours are inferred.

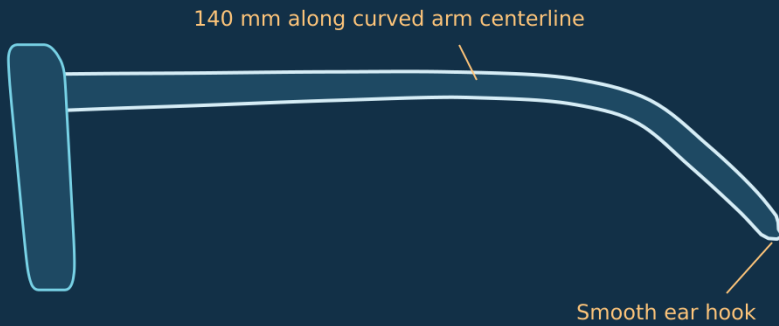
02 TOP / PLAN VIEW

-Z extends toward the ears · mild outward arm flare
140 front



03 SIDE ELEVATION

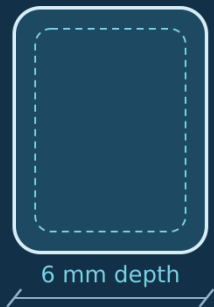
Flattened cross section · straight shaft → smooth ear hook



No cylindrical tube arms.
Front projection includes the gentle bow and 5° tilt.

04 EDGE / SECTION

Acetate profile; enlarged schematic



0.55 mm silhouette bevel
0.65 mm bevel depth
Clear lens thickness: 1.2 mm

COORDINATES

+X right · +Y up · +Z forward · origin at bridge

ASSEMBLY

Continuous front + 2 lenses + 2 pivoted arms + oval rivets + hinge pins

DESIGN ESTIMATE

One perspective photograph. Orthographic views and dimensions are inferred; not a manufacturing drawing.

Source geometry: design.json + glasses-model.js / Three.js r186

Open the package

- Open `glasses-workbench.html` in a browser. It embeds `Three.js`, the model, reference image and blueprint; it needs no CDN or build step. WebGL must be available.
- Drag to rotate; scroll to zoom; right-drag to pan. Choose Front, Side or Top for orthographic inspection.
- Use the sliders to adjust front width, arm length, finish, arm opening and assembly separation.
- Export GLB creates an assembled, unfolded model in meters. The inspection controls do not change the exported pose.
- `black-acetate-glasses.glb` is the ready-to-import model. `src/glasses-model.js` and `src/design.json` are the editable procedural source for an existing `Three.js` project.
- `glasses-blueprint.svg` is the editable vector drawing. `glasses-blueprint.png` is the high-resolution sheet. The drawings and JavaScript use the same contour data.

1. Read the photograph before choosing geometry

Preserve these visible features:

Feature	Reconstruction choice
Continuous glossy black front	One connected acetate mesh with two real lens holes
Brow thicker than lower rim	Independent outer and inner Bézier contours
Rounded rectangular openings	Custom curves; broader upper edge and narrower lower edge
Molded bridge	Front outline itself forms the bridge and nose opening
Small horizontal silver ovals	Flattened ellipsoids near the outer front corners
Flat, tapered arms	A rounded rectangular section swept along a 3D curve
Downturned ear ends	Curved centerline, with a tapered capped tip
Clear lenses	Separate gently bowed solid meshes with physical transmission

Do not create this frame from circular toruses or four rounded boxes. Those shapes cannot reproduce its brow, bridge and changing rim width. Do not trace the perspective image directly into a flat front plane: the nearer right lens appears larger because of the viewpoint. First establish a plausible symmetric front, then calibrate the camera against the image.

2. Establish scale and coordinates

All values below are design estimates. They are not measurements extracted from the photograph. They apply to the default 140 mm model.

Parameter	Starting value	Meaning
Overall front width	140 mm	Outside-to-outside acetate front
Front height	43.8 mm	Generated outer contour height
Lens opening	52 × 37 mm	Bounding size of each aperture
Closest horizontal lens gap	18 mm	Gap between aperture bounding boxes; not a confirmed marked bridge size
Acetate front depth	6 mm	Local solid depth, including bevels
Silhouette bevel	0.55 mm	Small rounding along outer and inner edges
Bevel depth	0.65 mm	Rounding toward front and rear surfaces
Clear lens thickness	1.2 mm	Approximate local solid thickness
Arm centerline length	140 mm	Arc length from hinge to tip, not straight-line depth
Arm section near hinge	3.2 × 6.5 mm	Thickness × height; tapers along the shaft
Front bow	4 mm	Outer edges recede from the center
Pantoscopic tilt	5°	Small tilt of the face; estimated
Front oval rivet	3.7 × 1.26 mm	Approximate visible ellipse

Use +X to the right when looking at the front, +Y upward, and +Z toward the viewer. The bridge plane is the model origin. Arms extend toward −Z. The source authoring dimensions are millimeters, but every returned geometry position and scene transform is in meters. A 140 mm frame therefore spans 0.140 scene units.

The width slider rescales the complete front in X, including the lens openings and their spacing. It preserves the design proportions rather than preserving a fixed optical lens size. The arm length slider changes centerline arc length around the same hinge positions.

3. Construct the continuous front



1. Define the right half of the outer silhouette using `outerRight` in `design.json`. Start at the bridge brow, travel around the outer wing and lower rim, and finish at the underside of the bridge.
2. Reverse and mirror that half to complete the left side. This keeps the bridge connected and avoids a seam made by attaching two independent rings.
3. Define the right opening with `innerRight`, normalize its bounds to 52×37 mm, and place its inner limit at $X = +9$ mm. Mirror it to the left.
4. Add both openings to `shape.holes`. Their winding must oppose the outer shape. The source checks winding with `ShapeUtils.isClockWise`.
5. Verify that every aperture point remains inside the outer outline. Pay particular attention to the inner nose edge: an intersecting hole can cause the triangulator to fill a lens opening with stray triangles.
6. Extrude the complete shape and bevel it. The supplied code uses a 4.7 mm core depth plus two 0.65 mm bevel depths, totaling 6 mm. `bevelOffset: -bevelSize` keeps the nominal outer silhouette size.

The official `ExtrudeGeometry` API supports shape extrusion, curve subdivisions and bevel controls [1]. The example below shows the authoring-unit settings; the source then deforms and converts the geometry to meters.

```
const coreDepth = 6 - 2 * 0.65;
const front = new THREE.ExtrudeGeometry(shape, {
  depth: coreDepth,
  steps: 1,
  curveSegments: 40,
  bevelEnabled: true,
  bevelSize: 0.55,
  bevelThickness: 0.65,
  bevelOffset: -0.55,
  bevelSegments: 4
});
front.translate(0, 0, -coreDepth / 2);
```

Apply a gentle bow and tilt before converting to meters:

```
z += -wrap * (x / (frontWidth / 2)) ** 2
  + y * Math.tan(THREE.MathUtils.degToRad(tilt));
position.setXYZ(i, x * 0.001, y * 0.001, z * 0.001);
```

Merge coincident vertices and recompute normals after deformation. The source removes UV attributes because it uses no image textures. If you add texture maps later, preserve and unwrap UVs rather than copying that step unchanged.

Edit the curves in this order: upper brow; outer wing; lower rim; inner nose edge; bridge arch. Keep the lower rim thinner than the brow. The aperture and outer contour must remain separate editable curves; a constant-width stroke loses the reference's changing rim thickness.

4. Add lenses without turning them into sunglasses



Reuse the lens opening curves to generate two separate solids. Expand each lens by 0.8% around its center so its edge is slightly buried in the acetate. The aperture is still 52 × 37 mm; the hidden lens mesh is slightly larger.

Use a thin beveled extrusion, then subdivide the broad faces before adding a gentle lens bow. Without interior subdivisions, bending only the perimeter leaves a few large flat triangles. This example uses `TessellateModifier` before deformation.

Use `MeshPhysicalMaterial` with transmission and keep opacity at 1, as specified by the official material documentation [2]. The source uses clear, untinted lenses; it does not reproduce a prescription.

```
const lensMaterial = new THREE.MeshPhysicalMaterial({
  color: 0xffffff,
  roughness: 0.025,
  transmission: 1,
  opacity: 1,
  ior: 1.5,
  thickness: 0.0012,
  envMapIntensity: 0.55
});
```

If the lenses appear gray, first inspect the reflected environment and transmission background. If one opening appears solid black, hide the lens mesh: a black area that remains indicates a front triangulation problem rather than a lens material problem.

5. Build flattened arms and hinge pivots



Use the eleven `templeControlPoints` in `design.json` as a centripetal Catmull-Rom centerline. They describe a mostly straight shaft, slight outward flare, and a smooth downturned tip. Mirror X for the second arm.

Measure the curve arc length with `getLength()` and scale its offsets from the hinge to achieve the selected 140 mm length. Do not equate this length with the plan-view depth.

Sample the curve along its length and build a local cross section at each point. Keep the section flat: approximately 3.2 mm thick and 6.5 mm high at the hinge, tapering toward about 2.5×4.5 mm along the shaft. The source uses a rounded rectangular superellipse rather than a circular `TubeGeometry`. Connect adjacent rings and cap both ends.

Attach each arm mesh to a group positioned at its hinge. `setOpen(90)` is fully open; reducing that angle folds the arms inward around opposite Y rotations. The small hinge pins and side ovals are visible approximations. Their exact hardware and hinge stops cannot be inferred from this image.

The assembly hierarchy is:

Parent	Children
Model root	front_frame; lens_L; lens_R; temple_pivot_L; temple_pivot_R
front_frame	front_rivet_L; front_rivet_R; hinge_pin_L; hinge_pin_R
temple_pivot_L	temple_L; temple_rivet_L
temple_pivot_R	temple_R; temple_rivet_R

Rivet positions, curves and hinge parts are named independently so you can inspect and refine them. The bridge is integral to `front_frame`; there is no added metal bridge or invented brand logo.

6. Match the finish and camera

Start with these materials, then adjust them after the silhouette matches.

Part	Color	Metalness	Roughness	Other
Acetate	#0b0d0f	0	0.20	Clearcoat 1; clearcoat roughness 0.14
Clear lenses	White	0	0.025	Transmission 1; IOR 1.5; thickness 1.2 mm
Silver details	#b7b3a6	0.88	0.25	Small, restrained reflections

Use an environment map for physical reflections [2]. The workbench generates one from Three.js RoomEnvironment, with a broad white light above-left and a faint ground shadow. A black material needs bright shapes to reflect; merely changing its base color cannot reproduce glossy acetate.

The supplied perspective camera is a starting comparison pose, not a recovered camera calibration: FOV 30°, position (0.115, 0.060, 0.250) m, target (0, -0.006, -0.049) m. The right lens appears larger and the arms recede toward the upper right, as in the reference. Front, side and top inspections use an orthographic camera.

For closer matching, adjust camera yaw first, elevation second, FOV/distance third, and framing last. Compare the front corner rivets, bridge center, lower lens corners, arm crest and near ear tip. Avoid warping one lens simply to imitate perspective.

Approximate reference landmarks, measured visually from the 384 × 341 image:

Landmark	Approximate pixel (X, Y)
Far front outer corner	(29, 145)
Far front oval rivet	(37, 153)
Bridge brow	(142, 171)
Near front oval rivet	(277, 184)
Near lower rim	(230, 257)
Near ear-tip end	(368, 189)

7. Use the source in your Three.js application

The procedural source is tested with Three.js 0.186.1. Copy both `src/glasses-model.js` and `src/design.json` into a project using a JSON-aware bundler such as your existing Vite setup. The source imports Three.js and its bundled addons; it does not load network textures.

```
import { createGlasses } from './src/glasses-model.js';

const glasses = createGlasses({
  frontWidth: 140,
  templeLength: 140,
  roughness: 0.20
});

scene.add(glasses.group);
glasses.setOpen(90);
glasses.setExploded(0);

// When replacing or removing this instance:
scene.remove(glasses.group);
glasses.dispose();
```

For the ready-made GLB:

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
const { scene: asset } = await new GLTFLoader()
  .loadAsync('/models/black-acetate-glasses.glb');
scene.add(asset); // already in meters
```

The workbench exports binary GLB with the official `GLTFExporter.parseAsync()` API [3]. Acetate clearcoat, lens transmission and volume properties are exported using supported glTF material extensions. Lighting and tone mapping belong to the receiving application, so another viewer may show different reflections.

The exported model includes only the glasses. It does not include the floor, camera, lights, reference image or blueprint. Export resets inspection separation and arm folding temporarily, then restores what you were viewing.

8. Refine toward an exact product match

1. Get measured front width, lens width/height, marked bridge size, arm length and acetate thickness. Replace the estimates before fine tuning.
2. Add a front photo with the camera centered, a top photo showing both arms, and a side photo showing the full ear hook. Use a ruler in the same plane as the part.
3. Fit the front silhouette and holes to the straight-on image. Check the bridge clearance and changing rim width.
4. Fit the arm shaft, hook and front bow to the top/side images. Replace approximate hinge pins with the measured hardware.
5. Match the original perspective image by tuning the camera. Compare silhouette overlap before judging highlights.
6. Only then adjust roughness, clearcoat, lighting and lens reflections. Retain subtle highlights; avoid turning the whole frame metallic or gray.

The included model is a detailed inspection version. For a mobile application, reduce curve and bevel subdivisions after the silhouette is accepted, lower shadow-map resolution, cap pixel ratio, and profile lens transmission on the target hardware. Physical transmission adds rendering cost [2]. Do not judge performance from the blueprint or from a screenshot.

Sources and package notes

[1] Three.js ExtrudeGeometry: <https://threejs.org/docs/pages/ExtrudeGeometry.html>

[2] Three.js MeshPhysicalMaterial: <https://threejs.org/docs/pages/MeshPhysicalMaterial.html>

[3] Three.js GLTFExporter: <https://threejs.org/docs/pages/GLTFExporter.html>

Reference image: the user's uploaded `image(20261006-042409).png`, copied unchanged as `reference.png`. Three.js is distributed under the MIT license; its license is included in `THIRD-PARTY-LICENSES.txt` and in the standalone workbench. The source creates all geometry procedurally, without external product assets or textures.